

CONTENTS

- Overview
- 1 Four kinds of intelligence, not a ladder
- 2 Humans: judgement, ambiguity and accountability
- 3 Rules and code: deterministic logic at scale
- 4 Where rules break down
- 5 Machine learning: patterns in structured data
- 6 What machine learning costs to keep running
- 7 Generative AI: unstructured input, flexible output
- 8 The real question: how much non-determinism can you tolerate?
- 9 Hybrid systems, and the heuristic
- Glossary
- Check yourself
- Where to go next

Choosing the Right Kind of Intelligence: Humans, Rules, ML and Generative AI

Learn to match a problem to the cheapest system that can actually solve it, and to recognise when AI is the wrong tool.

For engineers and technical leads deciding where AI belongs in a system and where it does not · 26 July 2026

[Watch the source video](#)[Read the condensed version](#)[Read the highlights](#)

BEFORE YOU START

- General software engineering experience
- A rough idea of what a trained ML model does
- Comfort reading Python

Overview

It is easy to fall into using AI for everything. Agent systems are genuinely powerful, and teams are wiring them across their stacks. But AI is not magic and it is not the best solution for every problem. Like any engineering decision, choosing whether to use it — and at what scale — is a trade-off across accuracy, cost, complexity and risk.

Most problems in software and data systems can be solved in one of four ways. Humans, for judgement and accountability. Rules or code, for deterministic logic. Machine learning, for statistical pattern recognition. Generative AI, for flexible reasoning over unstructured input. These are not a ladder you climb from human up to agent. They are distinct options, each with a specific role, and the skill is picking the right one.

The practical stakes are higher than they look. Most failures to reach production do not come from a bad model. They come from choosing the wrong kind of system at the start, and then spending months trying to make it behave like a different one.

KEY TAKEAWAYS

- ✓ Four options — humans, rules, ML, generative AI — are distinct tools, not rungs on a ladder.
- ✓ Humans are the right choice for high-stakes, ambiguous, accountability-bearing decisions, and they do not scale.
- ✓ Rules are the right choice when logic is known, explicit and stable, and when errors are unacceptable.
- ✓ Rules break when conditions change faster than you can update them, or when the logic grows too complex to write down.
- ✓ Machine learning fits when patterns exist in structured data but are not obvious enough to encode by hand.
- ✓ ML is not fire-and-forget: drift monitoring and retraining are permanent operating costs.
- ✓ Generative AI fits unstructured input, interpretation and synthesis — where flexibility matters more than exactness.
- ✓ The deciding question for generative AI is how much non-determinism you can tolerate, and the best systems combine all four.

01 Four kinds of intelligence, not a ladder

0:00

Humans, rules, machine learning and generative AI are distinct options with distinct roles. Choosing among them is an ordinary engineering trade-off.

AI agent systems are undeniably powerful, and it is easy to start integrating them everywhere. But it is worth staying strategic about where — and whether — to inject AI at all. Like any engineering decision, it involves trade-offs across accuracy, cost, complexity and risk.

Most problems in software and data systems can be solved in one of four ways. Humans, for judgement. Rules or code, for deterministic logic. Machine learning, for statistical pattern recognition. Generative AI systems, for flexible reasoning and generation.

The important framing is that these are not a ladder you progress up from humans to agents. They are distinct options, each with a very specific role, and the goal is to pick the right system for the right problem. Treating them as a progression is what produces the characteristic mistake of the moment: replacing a working rule with a model because the model feels more advanced, and inheriting non-determinism, cost and an untestable failure surface in exchange for nothing.

A good habit is to work from the bottom up. Start with the cheapest, most deterministic option that could possibly work, and move up only when you hit a specific, articulable wall. 'A rule cannot express this because the logic changes weekly' is a reason. 'This feels like an AI problem' is not.

KEY POINTS

- Four distinct options: humans, rules, ML, generative AI.
- They are alternatives with different strengths, not stages of sophistication.
- The trade-offs are accuracy, cost, complexity and risk — an ordinary engineering decision.
- Reach for the cheapest deterministic option that works, and escalate only on a specific wall.
- Replacing a working rule with a model buys non-determinism and cost for no gain.

The four options side by side

The row that decides most arguments is 'determinism'. Everything downstream — how you test, what you can guarantee, what happens on a bad day — follows from whether the same input always produces the same output.

	humans	rules	ML	generative AI
deterministic	no	yes	mostly	no
cost per call	very high	near zero	low	moderate to high
latency	minutes+	microsec	millisec	seconds
scales to	hundreds	unlimited	millions	limited by cost
interpretable	explains	fully	varies	post-hoc only
testable	no	exhaustive	statistical	hard
handles novelty	yes	no	poorly	yes
accountability	yes	designer	designer	designer

Escalate on a wall, not on a feeling

Each arrow needs a sentence you could defend in review. If you cannot write the sentence, you are not at the wall yet.

rules → ML	"the conditions change faster than we can ship rules"
rules → ML	"there are 400 interacting conditions and no one can state the logic"
ML → generative AI	"the input is unstructured text with no fixed schema"
anything → human	"someone has to be accountable if this is wrong"
# "it feels like an AI problem" is not a wall	

People handle incomplete and conflicting information, apply ethics and context, and can be held responsible. They are also expensive and do not scale.

Humans are very good at complex judgement. People handle ambiguity and edge cases well, making decisions from incomplete or conflicting information. They apply context, ethics and accountability in ways machines cannot.

So humans should drive tasks involving high-stakes decisions, situations requiring ownership or liability, and cases with significant ambiguity. Hiring decisions. Medical diagnosis. Legal interpretation. Large-scale strategy and financial decisions. If something goes wrong, accountability has to rest with a real person.

That last criterion is different in kind from the others, and it is the one most often overlooked. Accuracy, cost and latency are engineering properties you can measure and improve. Accountability is not a property of the system at all — it is a property of the arrangement around it, and no model improvement produces it. A model that is right ninety-nine percent of the time still cannot be the one held responsible for the other one percent.

The trade-off is that human judgement is high quality but expensive, slow and very hard to scale. Which is why the useful pattern is almost never all-or-nothing. Screening résumés automatically is sensible; the final hiring decision involves team fit and communication, which you would not want fully automated. Conversely, you would never have humans manually review every transaction on a credit card network — the volume is far past anything human review could touch. Automate the filtering, reserve the judgement.

KEY POINTS

- Humans handle ambiguity, conflicting information, context and ethics.
- Use them for high-stakes decisions, liability, and genuine ambiguity.
- Accountability is not an engineering property — no accuracy improvement creates it.
- The cost is expense, latency and an inability to scale.
- The usual answer is a split: automate the filtering, keep the judgement.

Splitting a decision by what it actually requires

Hiring, taken apart. The mechanical parts are automated and the judgement is preserved — and notice the automated stage is deliberately built to over-include, because a false negative here is invisible and unrecoverable.

```
def screen(applications):
    # rules: mechanical, auditable, and deliberately generous
    eligible = [a for a in applications
                 if a.work_authorised and a.years_experience >= 2]

    # ML: rank by similarity to past successful hires – a suggestion, not a gate
    ranked = model.rank(eligible)

    # humans: everything that carries consequence
    return ranked[:40]          # a person reads all forty

# the filter's job is to not lose good candidates, not to pick the winner
```

When volume settles the argument

Two cases at opposite ends. Volume alone can make human review impossible regardless of how much you would prefer it, which is what forces the automated system to exist in the first place.

hiring for one role	~200 applications	human review: feasible consequence: high, personal → automate filtering only
card transaction approval	millions	human review: impossible consequence: per-item small → rules + ML, humans on appeals

03 Rules and code: deterministic logic at scale

3:01

When the logic is known, stable and must not be wrong, ordinary code is fast, cheap, testable and completely interpretable.

A traditional software solution is good at executing clear, stable logic and producing consistent, exact outputs at scale. Rule-based systems are ideal when the logic is known and explicit — if X, do Y — when requirements are stable, and when errors are unacceptable.

The canonical cases are payment processing, input validation, data transformations and, most importantly, security and access control. Code here is fast, cheap, reliable and interpretable. It is straightforward to test, to debug and to scale.

To process a card payment you want a code path that checks whether the balance is sufficient and approves or declines. Fast, deterministic, no room for error. Using AI here introduces unnecessary risk and buys nothing. The same for validating an email address format: you do not need a model, you need a rule.

The property worth dwelling on is exhaustive testability. A rule can be tested against every branch, and once it passes it will behave identically forever. That is not a nice-to-have in the security and money domains — it is the requirement. A statistical system cannot offer it even in principle, and 'correct 99.99% of the time' is a different and much weaker claim than 'correct'.

The corollary is a rule you can state simply: anything that must never be wrong should not be probabilistic. Access control decisions, monetary arithmetic, data integrity constraints. A model may advise, flag, rank or explain around these. It should not be the thing that decides.

KEY POINTS

- Rules suit known, explicit, stable logic where errors are unacceptable.
- Payment processing, validation, data transformation, access control.
- Code is fast, cheap, interpretable, debuggable and exhaustively testable.
- A tested rule behaves identically forever; a model offers only a statistical claim.
- Anything that must never be wrong should not be probabilistic.

What deterministic actually buys you

Four assertions and the behaviour is pinned for the lifetime of the code. No equivalent exists for a model: you can measure accuracy on a sample, but you cannot assert on every input.

```
def authorise(account, amount):
    if amount <= 0:                return DECLINE("invalid amount")
    if account.frozen:             return DECLINE("account frozen")
    if amount > account.available: return DECLINE("insufficient funds")
    return APPROVE

def test_authorise():
    assert authorise(Account(available=100), 50) == APPROVE
    assert authorise(Account(available=100), 150) == DECLINE
    assert authorise(Account(available=100), 0) == DECLINE
    assert authorise(Account(frozen=True), 1) == DECLINE

# every branch covered. behaviour fixed forever. no drift, no monitoring,
# no retraining, no confidence interval.
```

Where a model may and may not sit

The distinction that keeps probabilistic systems out of trouble. Models are excellent at ranking, flagging and explaining. They should not be the component that issues the decision when the decision must be exact.

acceptable	not acceptable
rank which alerts to review first	decide whether to grant access
flag a transaction as suspicious	compute the account balance
draft an explanation of a decline	authorise the payment
suggest a validation rule	validate the input in production
summarise an audit log	enforce the retention policy

04 Where rules break down

3:44

Rules fail when conditions change constantly or the logic becomes too complex to state. Fraud detection is the standard example.

Rules break down where conditions change constantly and the logic becomes too complex to write out. Fraud detection is the textbook case. A static rule like 'flag any transaction over \$1,000' becomes useless quickly, because fraudulent actors adapt — the rule is public in effect the moment it is deployed, and the behaviour it targets simply moves.

There are two distinct failure modes worth separating, because they have different signatures. The first is adaptive adversaries: the environment responds to your rule, so any fixed threshold decays from the day it ships. The second is combinatorial complexity: no adversary at all, just so many interacting conditions that no person can hold the logic in their head or state it correctly.

Both produce the same symptom in a codebase, and it is easy to recognise once you have seen it. The rule set grows to hundreds of conditions with exceptions layered on exceptions. Nobody can predict what a change will do. The team becomes afraid to touch it. Each new case is handled by appending another special case rather than by revising the model of the problem.

That is the wall. When you hit it, the question stops being 'what is the right rule?' and becomes 'what property distinguishes these cases?' — which is a question a learned model can answer from examples and a person cannot answer from introspection.

KEY POINTS

- Adaptive adversaries make any fixed threshold decay from the moment it ships.
- Combinatorial complexity makes correct logic impossible to state, with no adversary involved.
- The symptom is the same: exception piled on exception, and fear of changing the code.
- Adding another special case treats the symptom rather than the cause.
- At the wall, the question changes from 'what is the rule?' to 'what distinguishes these cases?'

A rule set decaying in public

Each line was added after a real incident and was correct at the time. Together they encode a history of past attacks rather than a model of fraud — which is why they catch last quarter's patterns and miss this quarter's.

```
def is_fraud(txn):
    if txn.amount > 1000:                return True    # week 1
    if txn.amount > 900 and txn.foreign:  return True    # week 3
    if 995 < txn.amount < 1000:          return True    # week 5
    if txn.count_last_hour > 4:          return True    # week 6
    if txn.count_last_hour > 2 and txn.night: return True    # week 8
    if txn.amount < 5 and txn.new_merchant: return True    # week 9
    # ... 340 more lines, each from an incident review

# every rule was right when written. attackers now transact at 994.
# nobody on the team will touch this file.
```

The signals that you have hit the wall

Any two of these together mean the problem has outgrown explicit logic. It is a better trigger than a feeling, and it can be observed in the repository rather than argued about.

- ☐ the rule file only ever grows
- ☐ new rules are added after incidents, never derived from principle
- ☐ nobody can predict the effect of changing a threshold
- ☐ exceptions have exceptions
- ☐ two rules contradict and both are load-bearing
- ☐ the team routes around the file rather than editing it
- ☐ accuracy degrades steadily without anyone changing anything

05 Machine learning: patterns in structured data

4:29

Learn the distinguishing pattern from examples instead of stating it. Right when patterns exist but are not obvious enough to encode by hand.

Machine learning excels at learning patterns in structured data and making probabilistic predictions. It is the right tool when patterns exist but are not obvious, when rules are too complex to define manually, and when what you need is a prediction.

The common applications are fraud detection, customer churn prediction, demand forecasting and recommendation systems. Take demand forecasting: a human relies on intuition and carries bias; a rule-based solution is too simplistic to pick up the nuances of demand trends. Machine learning learns complex trends and patterns from the data, producing more accurate predictions — which makes it the right system for that particular task.

The key shift is what you supply. With rules you supply the logic. With ML you supply labelled examples and the model derives the logic. That is why it handles the fraud case: nobody needs to articulate what fraud looks like this month, because the pattern is learned from what was recently labelled as fraud.

It also relocates the difficulty. Your work becomes feature engineering and label quality rather than branching logic — and label quality is usually the harder problem. A fraud model trained on labels that only exist where the old rules fired will learn the old rules, including their blind spots. If you never labelled the transactions the rules missed, the model cannot learn to catch them, and it will look excellent in evaluation while inheriting exactly the gap you built it to close.

KEY POINTS

- ML suits structured data where patterns exist but are not obvious enough to state.
- You supply labelled examples; the model derives the logic.
- Standard applications: fraud, churn, demand forecasting, recommendations.
- The work moves from branching logic to feature engineering and label quality.
- Labels inherited from an old rule system teach the model that system's blind spots.

Rules versus a learned model on the same problem

The right-hand version never mentions a threshold. It learns which combinations of features separate the classes, including interactions nobody would think to write — and it is retrained rather than edited when behaviour shifts.

```
# rules: you state the logic
if txn.amount > 1000 and txn.foreign and txn.night:
    flag()

# ML: you supply examples and features, and the logic is derived
features = [
    "amount", "amount_vs_user_median", "hour_of_day",
    "seconds_since_last_txn", "merchant_risk_score",
    "distance_from_last_txn_km", "is_new_merchant_for_user",
]

model = GradientBoostingClassifier()
model.fit(X_train[features], y_train)      # y = labelled fraud/not fraud

score = model.predict_proba(txn_features)[0][1]    # 0.0 to 1.0
```

The label trap

The most common way a fraud model disappoints in production. Evaluation looks strong because the test set carries the same bias as the training set, and the model has learned to reproduce a system rather than to detect fraud.

```
# labels come from cases the OLD rules flagged and analysts confirmed

labelled fraud      : transactions the rules caught      ✓ in training data
labelled clean      : transactions the rules passed      ✓ in training data
unlabelled reality  : fraud the rules never flagged      ✗ invisible

# the model learns to reproduce the rules, blind spots included,
# and scores 0.97 AUC on a test set with the identical bias.

# fix: label a random sample independently of what the rules did,
# even though it is expensive and finds very little.
```

Where the probability threshold lives

A model outputs a score, not a decision. Turning it into an action is a business choice about the relative cost of the two error types — and it belongs in reviewable code, not buried inside the model.

```
score = model.predict_proba(txn)[0][1]

if score > 0.95: block()           # very likely fraud
elif score > 0.60: send_for_review() # a human decides
else:           approve()

# the thresholds encode what a false positive costs versus a false
# negative. that is a business decision, and it should be visible.
```

06 What machine learning costs to keep running

5:13

Models scale well and behave predictably, and they need monitoring, retraining and an honest account of what they can explain.

Machine learning models scale very well and are generally predictable. But they require more monitoring and ongoing maintenance than a rule does, because over time there is model drift, and because they have low flexibility outside the problem scope they were defined against.

Drift is the central operational fact. A model learns the relationship between inputs and outcomes as it stood in the training data, and the world moves on. Customer behaviour shifts, an upstream service changes a field's units, a competitor enters the market, a fraud pattern evolves. Accuracy degrades quietly, without any error and without anyone changing a line of code — which is why it needs to be watched rather than waited for.

Drift comes in two flavours worth distinguishing. Data drift is a change in the input distribution: the features arriving today no longer look like the ones the model trained on. Concept drift is a change in the relationship itself: the same inputs now imply a different outcome. Data drift is straightforward to detect by comparing distributions. Concept drift is harder, because you need outcomes to notice it, and outcomes often arrive weeks later — churn is only observable after the customer has gone.

Explainability also has real limits, and they are worth stating plainly before someone promises otherwise. Models have varying degrees of explainability, and the honest boundary is this: a model can tell you which features drove a prediction. It cannot tell you why the world works that way. Ask a demand forecaster for the societal causes of the trends it found and you are out of luck — it learned a correlation, and that is all it has.

KEY POINTS

- Models scale well and behave predictably, but require ongoing monitoring.
- Drift degrades accuracy silently, with no error and no code change.
- Data drift is a change in the inputs; concept drift is a change in the input-outcome relationship.
- Concept drift needs outcomes to detect, and outcomes often arrive late.
- Feature attribution explains which inputs drove a prediction, never why the world behaves that way.

Detecting data drift

Population Stability Index compares the live feature distribution against the training baseline. Run it per feature on a schedule; the conventional reading is that below 0.1 is stable, 0.1 to 0.2 warrants attention, and above 0.2 means retrain.

```
import numpy as np

def psi(baseline, current, bins=10):
    edges = np.percentile(baseline, np.linspace(0, 100, bins + 1))
    edges[0], edges[-1] = -np.inf, np.inf

    b = np.histogram(baseline, bins=edges)[0] / len(baseline)
    c = np.histogram(current, bins=edges)[0] / len(current)

    b, c = np.clip(b, 1e-6, None), np.clip(c, 1e-6, None)
    return float(np.sum((c - b) * np.log(c / b)))

for feature in FEATURES:
    score = psi(train_df[feature], live_df[feature])
    if score > 0.2:
        alert(f"{feature} drifted: PSI={score:.3f}")

# catches an upstream unit change on the day it happens, rather than
# three weeks later when someone notices revenue is down
```

What attribution can and cannot say

The left column is real, useful and defensible. The right column is what stakeholders usually want and what no model can supply — and confusing the two is how a forecasting model ends up cited in a strategy document it cannot support.

the model can say	the model cannot say
"price sensitivity contributed 0.31 to this prediction"	"customers are price-sensitive because of the recession"
"demand rises with this feature"	"demand rises because of this"
"these 5 features drive most of the variance"	"here are the societal causes of the trend"
# correlation learned from data. no causal claim is available.	

07 Generative AI: unstructured input, flexible output

5:53

Right when inputs are text or media, tasks need interpretation or transformation, and flexibility matters more than precision.

Analysing and predicting trends over massive structured data is not what a language model is for. What they are good at is unstructured data and tasks involving reasoning, synthesis and language.

Generative AI systems — LLMs and agents — are best used when inputs are unstructured, such as text, documents and other media; when tasks require interpretation or transformation; when flexibility matters more than precision; and when some amount of error is tolerable. Common cases are retrieval augmented generation for question answering over document sets, large document summarisation, code generation, and multi-step agent workflows.

Customer support is the clearest illustration. Instead of hard-coded responses to user questions, an agent can understand intent, pull relevant knowledge and generate a specific, useful reply. Doing that with rules alone would be close to impossible — the input is free text, the space of questions is unbounded, and no decision tree survives contact with how people actually write.

The distinguishing property is that the input has no schema. Rules and classical ML both require structured features: columns, numbers, categories. Generative models take the raw thing. When you find yourself writing regexes and parsers to force free text into a schema so a classifier can consume it, that parsing step is often the whole problem, and it is the part a language model handles well.

The upside is real: these systems are flexible and time-to-first-result is very fast, because a general model is a capable generalist before you have trained anything at all. That speed is genuine and it is also the trap — a prototype that works in an afternoon reveals nothing about what happens on the thousandth run.

KEY POINTS

- Generative AI suits unstructured input: text, documents, media.
- It suits interpretation, synthesis and transformation rather than numeric prediction.
- Use it where flexibility matters more than precision and some error is tolerable.
- The distinguishing property is that the input has no schema.
- Time to first result is very fast, which conceals how much work reliability will take.

Why rules cannot take this input

One support message containing three intents, a factual error, an implied deadline and an emotional register that should change the reply. There is no schema here, and every attempt to impose one loses something a human reader would act on.

```
"hi so we upgraded to the enterprise plan last tuesday i think? but the
sso thing still isnt working for half the team and my colleague said
something about a separate config?? also we were told this would include
priority support but nobody has replied in 3 days. we have an audit
friday so this is getting urgent"
```

```
# intents      : SSO fault, billing/entitlement query, support SLA complaint
# facts to check: upgrade date, whether SSO needs separate configuration
# implicit     : hard deadline on Friday
# register     : frustrated, escalating
```

```
# no parser recovers this. an LLM reads it the way a person does.
```

Extracting structure so the rest of the system can work

The most reliable production pattern. The model does the one thing it is uniquely good at — turning unstructured input into a schema — and everything after that is ordinary deterministic code that can be tested normally.

```
SCHEMA = {
    "intent": ["sso_fault", "billing", "sla_complaint"],
    "urgency": "high",
    "deadline": "2026-07-31",
    "account_claims": {"plan": "enterprise", "upgraded": "2026-07-21"},
}

ticket = model.extract(message, schema=TicketSchema) # generative

verify_plan(ticket.account_claims) # rules: check against billing
route(ticket.intent, ticket.urgency) # rules: deterministic routing
priority = sla_model.predict(ticket) # ML: predicted breach risk

# the model handles the part with no schema. everything else is testable.
```

08 The real question: how much non-determinism can you tolerate?

7:21

Generative systems do not guarantee correctness across runs, which makes them hard to test and costly at scale. That tolerance is the deciding criterion.

The trade-off with generative AI is non-determinism. The same input can produce different answers, which makes testing difficult, means correctness is not guaranteed across runs, and tends to be significantly more expensive at scale than the alternatives. That is really the key to deciding whether to build with agents and LLMs: how much non-determinism and uncertainty are you willing to tolerate?

Non-determinism breaks the testing model most engineers rely on. You cannot assert equality on the output. You cannot achieve branch coverage. A test that passes today can fail tomorrow with no change to your code — and worse, a test that passes can be passing for the wrong reason. What replaces exact assertions is a set of weaker but still useful techniques: assert on properties rather than strings, run each case several times and require a pass rate, keep a golden set and watch for regressions, and use a second model as a judge where the output is genuinely open-ended.

The honest framing is that you are not testing correctness any more. You are measuring a distribution and deciding whether its tail is acceptable.

Which is exactly why the arithmetic should not be inside the model. Ask an agent to analyse a year of spending and you cannot feed in thousands of transactions and expect a non-hallucinated answer, or even correct sums. Give it a calculator tool that uses code and maths for determinism, and traditional machine

learning to identify trends over time. Then let the LLM generate a clean natural-language report at the end. Each component does the part it can guarantee.

KEY POINTS

- The same input can produce different outputs, so exact-match testing does not apply.
- Correctness is not guaranteed across runs, and costs are higher at scale.
- Replace assertions with property checks, pass rates over repeated runs, golden sets and judge models.
- You are measuring a distribution and judging its tail, not verifying correctness.
- Arithmetic and aggregation belong in a tool, not in the model's head.

Testing something that will not sit still

Three assertions that survive non-determinism. Note the pass-rate test: running once tells you almost nothing, and a case that passes eight times in ten is a very different thing from one that passes always.

```
def test_extraction_properties():
    out = extract(MESSAGE)
    assert set(out.intents) <= VALID_INTENTS      # never invents a category
    assert out.deadline is None or is_iso_date(out.deadline)
    assert out.urgency in ("low", "medium", "high")

def test_pass_rate(n=20, threshold=0.9):
    passes = sum("sso_fault" in extract(MESSAGE).intents for _ in range(n))
    assert passes / n >= threshold, f"only {passes}/{n}"

def test_golden_set():
    scores = [grade(model_answer(c.input), c.expected) for c in GOLDEN]
    assert mean(scores) >= BASELINE - 0.02      # regression guard
```

Give the model a calculator

The spending-analysis example, decomposed. The model never performs arithmetic — it decides what to compute and how to describe the result, and code produces the numbers. This is the single most effective way to make a generative feature trustworthy.

```
def analyse_spending(transactions, year):
    # deterministic: code does the maths, exactly, every time
    totals = aggregate_by_category(transactions, year)
    monthly = monthly_series(transactions, year)

    # ML: trend detection over structured numeric data
    trends = trend_model.detect(monthly)
    anomalies = anomaly_model.flag(monthly)

    # generative: turn verified numbers into readable prose
    return model.generate(
        f"Write a short spending summary from these figures. "
        f"Use only the numbers given; do not compute anything.\n\n"
        f"{totals}\n{trends}\n{anomalies}"
    )

# hand an LLM 4,000 transactions and ask for the total and you will get
# a plausible number. it will be wrong, and it will look right.
```

09 Hybrid systems, and the heuristic

8:11

The best systems use all four in combination. 'Make everything agents' was never meant literally.

Hybrid systems tend to be the most successful — using all of these types in conjunction, balancing the right tool against each part of the task. When someone says make everything agents, that is not literal. All the best agents and modern solutions use some combination of these systems, and the work is identifying where each is appropriate.

The actionable heuristic is short. Use humans when you need judgement, accountability and ethics, or the workflow is high-risk. Use rules or traditional code when the logic is clearly defined — if X happens, do Y — and will not change often. Use traditional machine learning when you want patterns and predictions from past data. Use generative AI when you need to interpret, generate or reason over complex inputs and flexibility matters more than precision.

Apply it per component rather than per product. 'Should this be an AI system?' is almost always the wrong granularity, because a real system contains a dozen decisions and they rarely want the same answer. Fraud detection is ML for scoring, rules for the block threshold, humans for appeals, and possibly

generative AI for writing the customer notification. Four different technologies inside what a roadmap calls one feature.

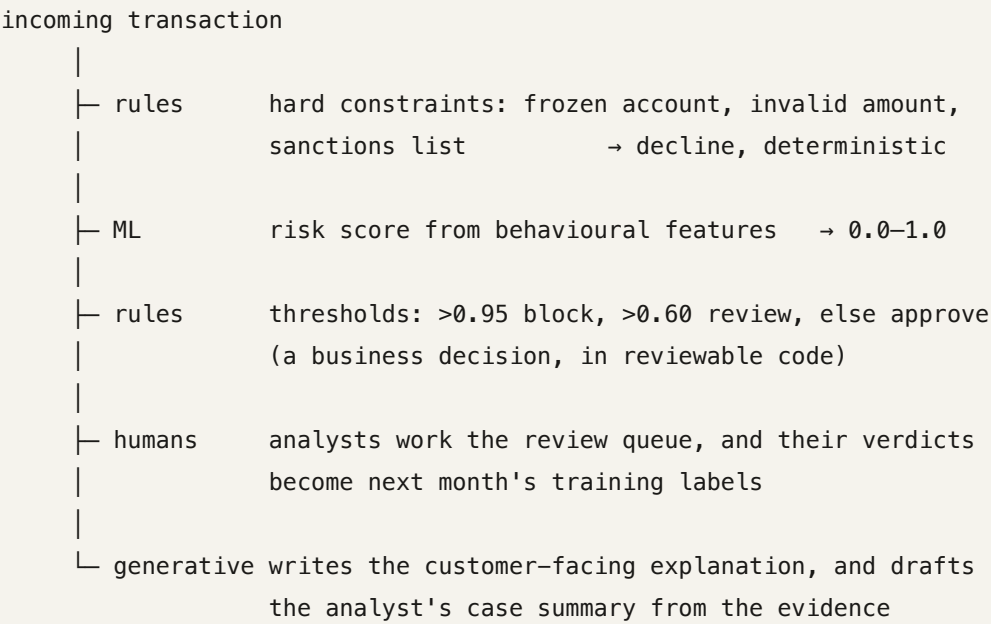
What is most important is matching the technology to the use case. Most failures to reach production do not come from bad models — they come from choosing the wrong system in the first place. The future is not about replacing humans or putting AI everywhere; it is about making disciplined choices about how decisions get made. The most successful AI systems will not be the ones using the most AI. They will be the ones that consistently pick the right kind of intelligence, which means the most important AI skill is knowing when not to use it.

KEY POINTS

- The strongest systems combine all four, each on the part it fits.
- Apply the choice per component, not per product.
- Humans for judgement and accountability; rules for stable explicit logic.
- ML for patterns in past structured data; generative AI for unstructured interpretation.
- Most production failures come from choosing the wrong system, not from a bad model.

One feature, four technologies

What 'we built AI fraud detection' actually contains. Each stage uses the cheapest system that can do its job, and the expensive, non-deterministic parts are kept off the path where correctness is required.



The heuristic, as a checklist

Work down the list and stop at the first match. It runs cheapest-first deliberately, because the failure mode this whole lesson is about is starting at the bottom.

Must it never be wrong, and is the logic statable? → rules
Must someone be accountable for the outcome? → humans
Is there a pattern in structured history you cannot write down by hand? → ML
Is the input unstructured, and is some error tolerable? → generative AI

None of the above cleanly? → you have more than one component.
split it and run the list again on each.

Questions to ask before adding an agent

Not obstacles — just the things that will be asked eventually, and are far cheaper to answer before the build than after. If several have no answer, the choice has not been made yet.

What does this cost per call at projected volume?
What happens when it is wrong, and who finds out?
How will we know it has degraded?
Can a rule handle 90% of cases, leaving the model the remaining 10%?
What is the deterministic fallback when the model is unavailable?
What is our tolerance for a different answer to the same question?

Glossary

Deterministic system

One where identical input always produces identical output. Exhaustively testable, and its behaviour is fixed once verified.

Rule-based system

Explicit conditional logic written by a person. Fast, cheap, interpretable and correct only for the cases anticipated.

Machine learning

Deriving a predictive function from labelled examples rather than stating it. Suits structured data with patterns too complex to encode by hand.

Model drift

Gradual degradation in accuracy as the world diverges from the training data. Silent: no error, no code change.

Data drift

A change in the distribution of incoming features relative to training. Detectable by comparing distributions.

Concept drift

A change in the relationship between inputs and outcomes. Harder to detect, because it requires outcomes that often arrive late.

Population Stability Index (PSI)

A measure of how much a feature's distribution has shifted from a baseline. Commonly read as stable below 0.1 and needing retraining above 0.2.

Non-determinism

The property that identical input may produce different output across runs. The defining cost of generative systems.

Golden set

A fixed collection of inputs with known-good outputs, scored repeatedly to catch regressions in a system you cannot assert on exactly.

LLM as judge

Using a second model to score an output against a rubric, where the output is too open-ended for a programmatic check.

Feature attribution

Identifying which inputs most influenced a prediction. Explains the model's behaviour, never the underlying causality.

Hybrid system

An architecture combining humans, rules, ML and generative AI, each applied to the part of the problem it fits.

Check yourself

Answer first, then open each one.

Why is 'accountability' a different kind of selection criterion from accuracy, cost or latency?

The others are measurable properties of the system that engineering effort can improve. Accountability is not a property of the system at all — it is an arrangement about who answers for the outcome, and no amount of model improvement produces it. A system that is right 99% of the time still cannot be responsible for the remaining 1%.

A team replaces a tested, working validation rule with a fine-tuned classifier. What have they traded away, and what have they gained?

They have given up exhaustive testability, deterministic behaviour, near-zero cost and full interpretability, and taken on drift monitoring and retraining. They gained nothing, because the logic was already known, explicit and stable — which is exactly the case rules are best at. This is the characteristic error of treating the four options as a ladder.

A fraud model scores 0.97 AUC in evaluation and performs poorly in production. What is the most likely cause, and why does evaluation not reveal it?

The training labels came from cases the old rule system flagged, so the model learned to reproduce those rules including their blind spots. Evaluation does not reveal it because the test set carries the identical bias — fraud the rules never caught was never labelled and appears in neither split. The fix is to label a random sample independently of what the rules did.

Why is concept drift harder to detect than data drift?

Data drift is a change in the input distribution, which you can detect immediately by comparing incoming features against the training baseline. Concept drift is a change in the input-to-outcome relationship, so detecting it requires knowing the outcomes — and those often arrive weeks later. Churn is only observable once the customer has already gone.

Why should an agent asked to analyse a year of transactions never compute the totals itself?

Because it produces a plausible number rather than a computed one, and a plausible total is indistinguishable from a correct one at a glance. Aggregation is exactly the deterministic work code guarantees, so the model should decide what to compute and describe the result while a tool produces the figures.

Why is 'should this feature use AI?' usually the wrong question?

Because a feature contains many decisions and they rarely want the same technology. Fraud detection alone wants rules for hard constraints, ML for scoring, rules again for thresholds, humans for appeals and generative AI for the customer notification. The choice belongs at the component level, and asking at the product level forces one answer onto four different problems.

Where to go next

- Take one feature you already ship and decompose it into components, labelling each as human, rule, ML or generative — most teams find at least one component using a heavier technology than it needs.
- Implement the PSI check over your production model's features and backfill it against the last six months of data to see whether drift you never noticed was already happening.
- Take a generative feature with no tests and write the three test shapes from this lesson — property assertions, a pass rate over repeated runs, and a golden set with a regression threshold.
- Audit any rule file that has grown past a few hundred lines against the wall checklist, and see whether the answer is a learned model or simply a rewrite.
- For a fraud or abuse model you own, check where the training labels came from; if they came from the previous rule system, estimate what fraction of real cases could never have been labelled.
- Find a place where an LLM is doing arithmetic or aggregation and replace it with a tool call, then compare outputs on a hundred cases — the error rate is usually higher than anyone expected.

Lesson generated from a YouTube transcript with YouTube Lesson Builder.

Explanations are AI-generated. Check anything you intend to rely on.